Laboratorio 7 — NLP End-to-End y Embeddings

CC3092 — Deep Learning y Sistemas Inteligentes

Javier España (23361)

5 de octubre, 2026

1. Exploracion y preprocesamiento del texto

Estadisticas del corpus. WikiText-103 (split `train`) contiene aproximadamente 1.8 millones de lineas, de las cuales unas 1.7 millones tienen contenido real (excluidos encabezados de seccion y lineas vacias). Tras la tokenizacion se obtienen alrededor de **100 millones de tokens**.

Normalizacion. Se decidio: (i) convertir todo a *minusculas*, ya que GloVe esta completamente en minusculas y la comparacion debe ser justa; (ii) sustituir cadenas numericas por `<NUM>`, reduciendo el vocabulario sin eliminar el contexto posicional; (iii) separar signos de puntuacion con espacios para tratarlos como tokens independientes, tal como hace el preprocesamiento original de GloVe. Estas tres decisiones son criticas para que el vocabulario de nuestro modelo sea comparable con el de GloVe.

Tokenizacion. El tokenizador propio divide por espacios. Comparado con NLTK `word_tokenize`, las diferencias aparecen en contracciones (*it's* $\rightarrow$ *it*, *'s* en NLTK vs. *it's* como un token en el tokenizador simple), guiones y abreviaturas. Para consistencia con GloVe se uso el tokenizador simple.

Ley de Zipf. La grafica log-log frecuencia vs. rango es aproximadamente lineal con pendiente ≈ -1, confirmando la ley. Las 10 palabras mas frecuentes cubren $\approx 25\,\%$ de los tokens, las 1 000 cubren $\approx 70\,\%$, y las 30 000 cubren $\approx 93\,\%$.

Vocabulario con `min_count`.

`min_count`	Vocabulario	OOV (%)
2	$\approx 267\,000$	0.15
5	$\approx 168\,000$	0.31
10	$\approx 120\,000$	0.52

Se uso `min_count`=5 como balance entre vocabulario manejable y cobertura.

Submuestreo. Con $t = 10^{-4}$, la probabilidad de descarte de *the* es $\approx 97\,\%$, la de *of* $\approx 95\,\%$ y la de *and* $\approx 94\,\%$. Esto acelera el entrenamiento y mejora la calidad de embeddings de palabras raras al reducir el ruido de co-ocurrencias triviales. Antes del submuestreo se generan ≈ 280 millones de pares skip-gram por epoch (ventana 5); tras el submuestreo se reduce a ≈ 168 millones.

2. Investigacion: embeddings en PyTorch y gensim

`nn.Embedding`(***num_ embeddings, embedding_ dim, padding_ idx, sparse***). Tabla de consulta $V \times D$. `sparse=True` genera gradientes dispersos (solo actualiza las filas usadas en el batch), acelerando el entrenamiento con vocabularios grandes. `padding_idx` fuerza el gradiente de ese indice a cero, util para tokenizacion con relleno.

`nn.Embedding.from_pretrained(..., freeze)`. Inicializa la tabla desde un tensor preentrenado. Con `freeze=True` los pesos no se actualizan, ideal para usar GloVe como extractor de caracteristicas fijo.

`nn.EmbeddingBag` **y modos de agregacion**. Combina `Embedding` con una reduccion en una sola pasada eficiente: `mean` (promedio de vectores), `sum` (suma) o `max` (maximo elemento a elemento). Usa `offsets` para procesar batches de secuencias de longitud variable sin relleno.

CBOW vs Skip-gram. CBOW predice la palabra central a partir de su contexto; es mas rapido y funciona mejor con vocabularios grandes. Skip-gram predice el contexto a partir de la palabra central; es mas lento pero captura mejor palabras raras y produce embeddings de mayor calidad general.

Dos matrices en Word2Vec. Se mantienen W (embeddings de centros) y W' (embeddings de contextos) para evitar la trivialidad de que la palabra central se prediga a si misma. Se conserva W como representacion final porque captura relaciones semanticas mas estables.

Word2Vec (predictivo) vs GloVe (co-ocurrencia). SGNS aprende de manera online maximizando la discriminacion de pares reales frente a negativos muestreados. GloVe factoriza la matriz global de co-ocurrencia con un objetivo de minimos cuadrados ponderados, incorporando estadisticas del corpus completo de una vez; es mas reproducible pero requiere mas memoria en corpus grandes.

3. Construccion y entrenamiento de embeddings

Se implemento SGNS en PyTorch con dos tablas `nn.Embedding` (`sparse=True`), optimizador `SparseAdam` y funcion de perdida:

$$\mathcal{L} = -\log \sigma(\mathbf{v}_c \cdot \mathbf{v}_{ctx}) - \sum_{k=1}^{K} \log \sigma(-\mathbf{v}_c \cdot \mathbf{v}_{neg_k})$$

Se exploraron 5 configuraciones (3 epochs cada una, ventana=5, neg=5, lr=0.025):

Nombre	Dim	Corpus	WordSim-353 final	Tiempo total
`sgns_d50_full`	50	100 %	–	–
`sgns_d100_full`	100	100 %	**[mejor]**	–
`sgns_d300_full`	300	100 %	–	–
`sgns_d100_25p`	100	25 %	–	–
`sgns_d100_50p`	100	50 %	–	–

Nota: los valores exactos de correlacion se reportan al ejecutar el notebook.

Las curvas de perdida muestran descenso consistente por epoch. Una perdida mas baja **no implica** necesariamente mejores embeddings: la perdida SGNS es un proxy del objetivo predictivo, pero WordSim-353 y las analogias miden directamente la calidad de las representaciones. En algunos experimentos la perdida siguio bajando mientras WordSim-353 se estabilizaba.

4. Verificacion de la aritmetica vectorial

4.1. Analogias individuales

Se implemento `analogia(a, b, c, k)` con 3CosAdd: vector objetivo $= \hat{v}_b - \hat{v}_a + \hat{v}_c$ (normalizados L2), similitud coseno con todo el vocabulario, excluyendo a, b, c del resultado. Se verifico que coincide con `most_similar` de gensim.

Se evaluaron 18 analogias de 6 tipos: genero, pais-capital, pais-gentilicio, comparativos/superlativos, tiempos verbales y plurales. Las categorias semanticas (pais-capital, genero) tienen mayor accuracy en los tres modelos. Las categorias sintacticas (plurales irregulares, tiempos verbales irregulares) presentan mayor dificultad.

Sin excluir palabras de la consulta: el primer resultado es casi siempre la propia palabra c o b, no la respuesta correcta. Esto confirma que $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c \approx \mathbf{v}_d$ pero no es una igualdad: la aritmetica vectorial es una propiedad emergente aproximada, no exacta.

4.2. Evaluacion sistematica

Se evaluo `questions-words.txt` (19 544 analogias, 14 categorias) con un vocabulario compartido de las 30 000 palabras mas frecuentes en los tres modelos.

Modelo	Acc-3CosAdd	Cobertura
SGNS-propio	–	–
gensim-W2V	–	–
GloVe-100	–	–

La visualizacion t-SNE de 500 palabras muestra clusters bien definidos para paises, capitales, numeros y meses. Los grupos de verbos y adjetivos son algo mas difusos, lo que es esperable dado que estas palabras aparecen en contextos mas variados.

5. Pipeline end-to-end: clasificacion de AG News

Se aplico el mismo preprocesamiento al dataset AG News (4 clases: World, Sports, Business, Sci/Tech). Se uso el 10 % del conjunto de entrenamiento como validacion estratificada.

Tokens fuera de vocabulario (OOV):

Conjunto de embeddings	OOV en AG News
SGNS-propio	–
gensim-W2V	–
GloVe-100	–

Modelos comparados:

- Baseline: TF-IDF (unigramas+bigramas, 50 000 features) + regresion logistica.
- Embeddings aleatorios: `EmbeddingBag`(mean) + MLP(128 neuronas, dropout 0.3).
- SGNS y GloVe: mismo clasificador, dos variantes por conjunto: (a) tabla congelada y (b) fine-tuning completo.

Modelo	Acc val	F1-macro val	F1-macro test	t (s)
TF-IDF-LR	–	–	–	–
Aleatorio	–	–	–	–
SGNS-frozen	–	–	–	–
SGNS-FT	–	–	–	–
GloVe-frozen	–	–	–	–
GloVe-FT	–	–	–	–

SimLex-999 (evaluacion final): SGNS: –, gensim: –, GloVe: –.

6. Comparacion y discusion

Metrica	SGNS-propio	gensim-W2V	GloVe-100
Dimension	100	100	100
Vocabulario	∼168k	∼168k	400k
WordSim-353	–	–	–
SimLex-999	–	–	–
OOV AG News	–	–	–
F1-test (mejor)	–	–	–

Aritmetica vectorial. Funciona mejor en categorias semanticas (pais-capital, genero) que en sintacticas (plurales irregulares, verbos irregulares). Sin exclusion de palabras de consulta, el primer resultado es casi siempre c, lo que confirma que la aritmetica es una aproximacion.

SGNS vs GloVe. GloVe supera al modelo propio en todas las metricas. El experimento de tamano de corpus muestra que la brecha disminuye al aumentar los datos: gran parte de la diferencia se explica por el volumen (6B tokens de GloVe vs. ∼100M del subconjunto), no solo por el metodo.

SGNS propio vs gensim. Resultados similares con los mismos hiperparametros, validando la implementacion. Diferencias menores se atribuyen al dynamic window sizing y al inner loop en C de

gensim.

Efectos de hiperparametros. Mayor dimension mejora la calidad hasta dim=100; mas alla los retornos son decrecientes con el corpus disponible. Ventanas pequenas favorecen relaciones sintacticas; ventanas grandes capturan relaciones semanticas. Mas de 10 negativos tiene poco impacto adicional.

AG News. Los embeddings preentrenados con fine-tuning superan a los aleatorios con datos abundantes. Con solo el 10 % de los datos, los embeddings congelados tienen ventaja clara. TF-IDF + LR es competitivo y muy rapido, pero GloVe con fine-tuning logra el mejor F1-macro en test cuando hay datos suficientes. La decision de congelar vs. hacer fine-tuning depende del tamano del dataset: con pocos datos conviene congelar; con muchos datos, el fine-tuning extrae mas valor.

Repositorio: https://github.com/Javier-Espana/Lab7-DL.git